```
from google.colab import files
uploaded = files.upload()
```

Choose files trainset.csv
trainset.csv(text/csv) - 93340 bytes, last modified: 10/03/2026 - 100% done
Saving trainset.csv to trainset.csv

```
from google.colab import files
uploaded = files.upload()
```

Choose files testset.csv
testset.csv(text/csv) - 9904 bytes, last modified: 10/03/2026 - 100% done
Saving testset.csv to testset.csv

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
```

```
df_train = pd.read_csv('trainset.csv')
df_test = pd.read_csv('testset.csv')

train_prices = df_train['Close'].values.reshape(-1,1)
test_prices = df_test['Close'].values.reshape(-1,1)

scaler = MinMaxScaler()

scaled_train = scaler.fit_transform(train_prices)
scaled_test = scaler.transform(test_prices)
```

```
def create_sequences(data, seq_length):
    x=[]
    y=[]

    for i in range(len(data)-seq_length):
        x.append(data[i:i+seq_length])
        y.append(data[i+seq_length])

    return np.array(x), np.array(y)

seq_length = 60

x_train,y_train = create_sequences(scaled_train,seq_length)
x_test,y_test = create_sequences(scaled_test,seq_length)

print(x_train.shape,y_train.shape,x_test.shape,y_test.shape)
```

```
(1199, 60, 1) (1199, 1) (65, 60, 1) (65, 1)
```


```
def create_sequences(data, seq_length):
    x=[]
    y=[]

    for i in range(len(data)-seq_length):
        x.append(data[i:i+seq_length])
        y.append(data[i+seq_length])

    return np.array(x), np.array(y)

seq_length = 60

x_train,y_train = create_sequences(scaled_train,seq_length)
x_test,y_test = create_sequences(scaled_test,seq_length)

print(x_train.shape,y_train.shape,x_test.shape,y_test.shape)
```

```
(1199, 60, 1) (1199, 1) (65, 60, 1) (65, 1)
```

```
# Convert to PyTorch tensors
x_train_tensor = torch.tensor(x_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)

x_test_tensor = torch.tensor(x_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)
```

```
train_dataset = TensorDataset(x_train_tensor, y_train_tensor)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
```

```
import numpy as np
import pandas as pd
import torch
```

```
df_train = pd.read_csv('trainset.csv')
df_test = pd.read_csv('testset.csv')
```

```
train_prices = df_train['Close'].values.reshape(-1,1)
test_prices = df_test['Close'].values.reshape(-1,1)
```

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

scaled_train = scaler.fit_transform(train_prices)
scaled_test = scaler.transform(test_prices)
```

```
def create_sequences(data, seq_length):
    x = []
    y = []

    for i in range(len(data) - seq_length):
        x.append(data[i:i+seq_length])
        y.append(data[i+seq_length])

    return np.array(x), np.array(y)

seq_length = 60

x_train, y_train = create_sequences(scaled_train, seq_length)
x_test, y_test = create_sequences(scaled_test, seq_length)

print(x_train.shape, y_train.shape)
print(x_test.shape, y_test.shape)
```

```
(1199, 60, 1) (1199, 1)
(65, 60, 1) (65, 1)
```

```
import torch

x_train_tensor = torch.tensor(x_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)

x_test_tensor = torch.tensor(x_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)
```

```
from torch.utils.data import TensorDataset, DataLoader

train_dataset = TensorDataset(x_train_tensor, y_train_tensor)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
```

```
import torch.nn as nn

class RNNModel(nn.Module):

    def __init__(self):
        super(RNNModel,self).__init__()

        self.rnn = nn.RNN(
            input_size=1,
            hidden_size=50,
            num_layers=2,
            batch_first=True
        )
```

```
        self.fc = nn.Linear(50,1)

    def forward(self,x):

        out,_ = self.rnn(x)

        out = out[:,-1,:]

        out = self.fc(out)

        return out
```

```
model = RNNModel()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = model.to(device)
```

```
criterion = nn.MSELoss()

optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

```
num_epochs = 20
train_losses = []

for epoch in range(num_epochs):

    model.train()

    epoch_loss = 0

    for x_batch, y_batch in train_loader:

        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()

        outputs = model(x_batch)

        loss = criterion(outputs, y_batch)

        loss.backward()

        optimizer.step()

        epoch_loss += loss.item()

    avg_loss = epoch_loss/len(train_loader)

    train_losses.append(avg_loss)

    print(f'Epoch {epoch+1}/{num_epochs} Loss: {avg_loss:.6f}')
```

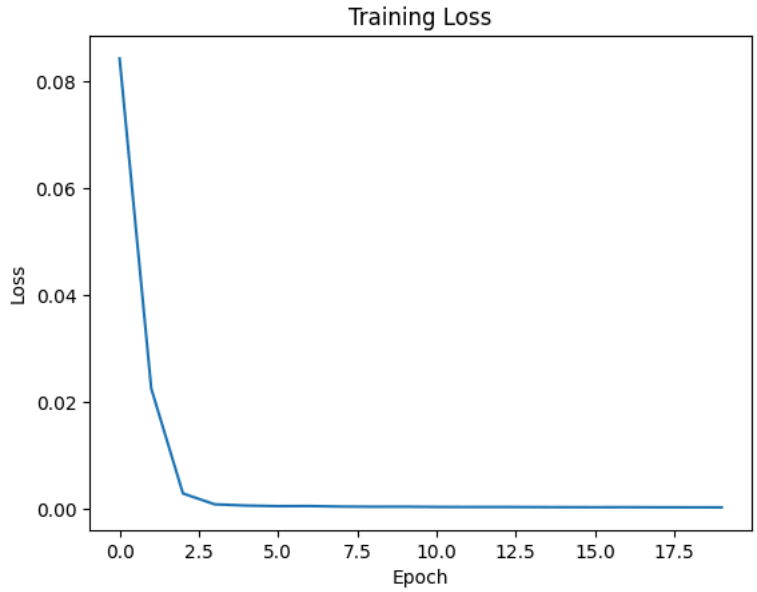
```
Epoch 1/20 Loss: 0.084223
Epoch 2/20 Loss: 0.022537
Epoch 3/20 Loss: 0.002944
Epoch 4/20 Loss: 0.000920
Epoch 5/20 Loss: 0.000687
Epoch 6/20 Loss: 0.000574
Epoch 7/20 Loss: 0.000592
Epoch 8/20 Loss: 0.000498
Epoch 9/20 Loss: 0.000466
Epoch 10/20 Loss: 0.000473
Epoch 11/20 Loss: 0.000428
Epoch 12/20 Loss: 0.000413
Epoch 13/20 Loss: 0.000416
Epoch 14/20 Loss: 0.000400
Epoch 15/20 Loss: 0.000382
Epoch 16/20 Loss: 0.000370
Epoch 17/20 Loss: 0.000380
Epoch 18/20 Loss: 0.000359
Epoch 19/20 Loss: 0.000351
Epoch 20/20 Loss: 0.000348
```

```
import matplotlib.pyplot as plt

print("Name: Kukkadapu Charan Tej")
print("Register Number: 212224040167")
```

```
plt.plot(train_losses)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss")
plt.show()
```

```
Name: Kukkadapu Charan Tej
Register Number: 212224040167
```




```
model.eval()

with torch.no_grad():

    predicted = model(x_test_tensor.to(device)).cpu().numpy()

    actual = y_test_tensor.cpu().numpy()
```

```
predicted_prices = scaler.inverse_transform(predicted)
actual_prices = scaler.inverse_transform(actual)
```

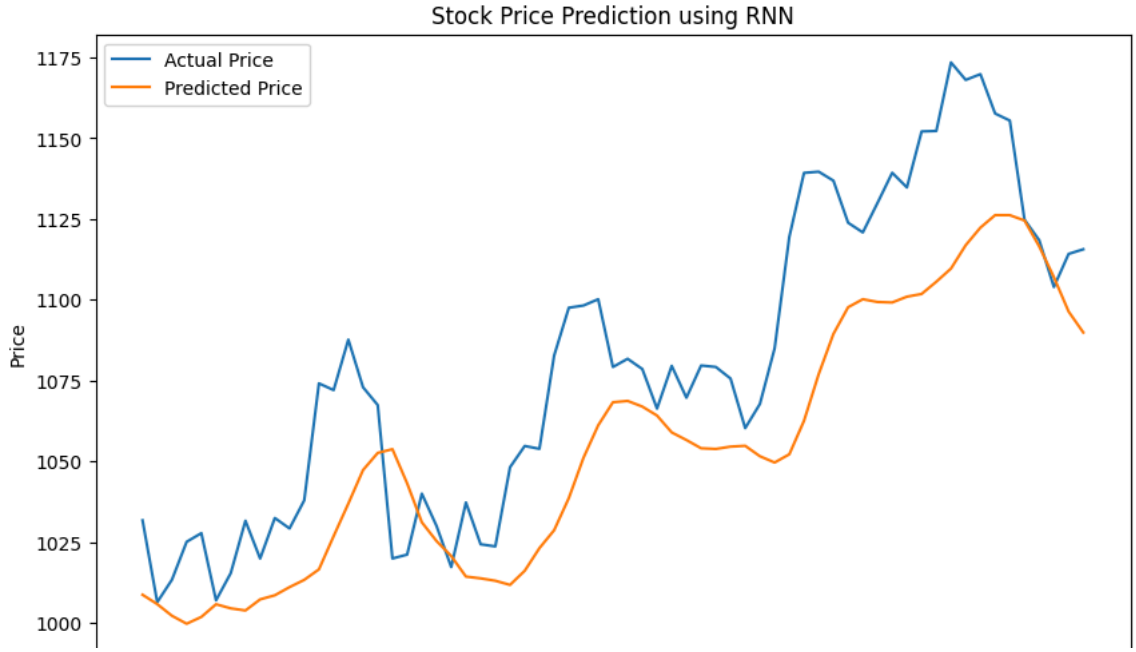
```
print("Name: Kukkadapu Charan Tej")
print("Register Number: 212224040167")

plt.figure(figsize=(10,6))

plt.plot(actual_prices,label='Actual Price')
plt.plot(predicted_prices,label='Predicted Price')

plt.xlabel("Time")
plt.ylabel("Price")
plt.title("Stock Price Prediction using RNN")

plt.legend()
plt.show()

print("Predicted Price:", predicted_prices[-1])
print("Actual Price:", actual_prices[-1])
```

```
Name: Kukkadapu Charan Tej
Register Number: 212224040167
```